

Below are **short, crisp, exam-ready notes** followed by **numerical / calculation-based problems with solved answers** for **each topic**.

I've kept them **precise, specific, and scoring-oriented**, the way examiners like.

1. Virtualization

Short Notes

- Virtualization allows **multiple virtual machines (VMs)** to run on a **single physical machine**.
 - Achieved by **abstracting hardware resources** (CPU, RAM, Storage, Network).
 - Benefits:
 - Better resource utilization
 - Isolation
 - Scalability
 - Reduced cost
 - Types:
 - **Full virtualization**
 - **Para-virtualization**
 - **OS-level virtualization**
-

Numerical Problem

Problem:

A server has:

- 16 CPU cores
- 64 GB RAM

Each VM requires:

- 2 CPU cores
- 8 GB RAM

How many VMs can be created?

Solution:

- CPU limit = $16 / 2 = 8$ **VMs**

- RAM limit = $64 / 8 = 8$ VMs

✓ **Maximum VMs = 8**

2. VMotion (Live Migration)

Short Notes

- VMotion migrates a **running VM** from one physical host to another.
 - No service downtime.
 - Requires:
 - Shared storage
 - High-speed network
 - Compatible CPUs
 - Used for:
 - Load balancing
 - Maintenance
 - Fault tolerance
-

Numerical Problem

Problem:

VM memory size = **8 GB**

Network bandwidth = **1 Gbps**

Time to transfer memory (ignore overhead)?

Solution:

- $8 \text{ GB} = 64 \text{ Gb}$
- $\text{Time} = 64 / 1 = 64 \text{ seconds}$

✓ **Migration time ≈ 64 seconds**

3. Hypervisor

Short Notes

- Software layer that enables virtualization.

- Controls and allocates hardware to VMs.

Types:

1. Type-1 (Bare-metal)

- Runs directly on hardware
- Faster, secure
- Example: ESXi, Hyper-V

2. Type-2 (Hosted)

- Runs on host OS
- Slower
- Example: VirtualBox

Numerical Problem

Problem:

Type-2 hypervisor introduces **15% performance overhead**.

Bare hardware executes a task in **200 ms**.

Execution time on Type-2 hypervisor?

Solution:

- Overhead = 15% of 200 = 30 ms
- Total time = 200 + 30 = **230 ms**

✅ Execution time = 230 ms

4. Hypervisor Manager

Short Notes

- Manages **multiple hypervisors and VMs centrally**.
- Handles:
 - VM provisioning
 - Monitoring
 - Load balancing
 - VMotion
- Examples:

- VMware vCenter
- Microsoft SCVMM
- OpenStack

Numerical Problem

Problem:

A manager monitors **40 hosts**, each with **25 VMs**.

Total VMs managed?

Solution:

[
 $40 \times 25 = 1000$
]

✓ **Total VMs = 1000**

5. Fault Tolerance

Short Notes

- Ensures system continues operation despite failures.
- Achieved using:
 - Replication
 - Checkpointing
 - Redundancy
- In virtualization:
 - Primary VM + Secondary VM
 - Instant failover

Numerical Problem

Problem:

Failure rate of one server = **0.01**

Two servers in **replicated fault-tolerant setup**

System failure probability?

Solution:

[
 $P(\text{system fails}) = 0.01 \times 0.01 = 0.0001$
]

✓ **System failure probability = 0.01%**

6. CUDA (Compute Unified Device Architecture)**Short Notes**

- NVIDIA's parallel computing platform.
 - Allows CPU to offload computation to **GPU**.
 - Uses:
 - Threads
 - Blocks
 - Grids
 - Best for:
 - Matrix operations
 - ML
 - Image processing
-

Numerical Problem**Problem:**

A CUDA kernel launches:

- 256 threads per block
- 80 blocks

Total threads launched?**Solution:**

[
 $256 \times 80 = 20,480$
]

✓ **Total threads = 20,480**

One-Line Exam Killers

- **Virtualization:** Hardware abstraction for multiple OS instances.
 - **VMotion:** Live VM migration with zero downtime.
 - **Hypervisor:** Software enabling virtualization.
 - **Hypervisor Manager:** Centralized VM control platform.
 - **Fault Tolerance:** Continuous service despite failures.
 - **CUDA:** GPU-based parallel computing model.
-

If you want, I can:

-  Convert this into **2-page exam notes**
-  Create **MCQs + numericals**
-  Provide **diagram-based explanations**
-  Give **past-paper style questions**

Just say the word 

Below are **short, concise, exam-ready notes** plus **numerical / calculation-based solved problems** for each topic.

I've kept answers **precise, technical, and scoring-focused**, exactly what is expected in **Cloud / Distributed Systems / Parallel Computing exams**.

1. MapReduce

Short Notes

- Programming model for **large-scale data processing**.
 - Works in **two main phases**:
 - **Map** → transforms input into key-value pairs
 - **Reduce** → aggregates values of same keys
 - Runs on distributed clusters (e.g., Hadoop).
 - Fault-tolerant and scalable.
-

Numerical Problem

Problem:

Input file size = **640 MB**

HDFS block size = **128 MB**

How many map tasks will be created?

Solution:

[
 $640 / 128 = 5$
]

✓ Map tasks = 5

2. MPI Broadcast (MPI_Bcast)

Short Notes

- One process (root) sends **same data to all processes**.
 - Communication pattern: **1 → N**
 - Efficient tree-based communication used internally.
-

Numerical Problem

Problem:

Broadcast a **32 KB** message to **15 processes**.

Total data sent from root (logical view)?

Solution:

[
 $32 \text{ KB} \times (15 - 1) = 448 \text{ KB}$
]

✓ Total data = 448 KB

3. MPI Scatter (MPI_Scatter)

Short Notes

- Root process splits data into **equal chunks**.
- Each process receives **different portion**.
- Communication pattern: **1 → N (partitioned)**

Numerical Problem

Problem:

Root has an array of **400 integers**.

Total processes = **8**

How many integers does each process receive?

Solution:

[
 $400 / 8 = 50$
]

✓ Each process gets 50 integers

4. MPI Gather (MPI_Gather)

Short Notes

- Opposite of Scatter.
 - Each process sends its local data to the root.
 - Communication pattern: **N → 1**
-

Numerical Problem

Problem:

Each of **10 processes** sends **200 bytes** to root.

Total data received at root?

Solution:

[
 $10 \times 200 = 2000 \text{ bytes}$
]

✓ Total received = 2000 bytes

5. MPI Reduce (MPI_Reduce)

Short Notes

- Combines data from all processes using an operation.
- Common operations:

- SUM
 - MAX
 - MIN
 - PROD
- Result stored at **root process**.
-

Numerical Problem

Problem:

Local values at 6 processes:

Process Value

P0	4
P1	6
P2	10
P3	2
P4	8
P5	5

Operation: **MPI_SUM**

Final result at root?

Solution:

[
 $4 + 6 + 10 + 2 + 8 + 5 = 35$
]

 **Reduce result = 35**

Another MPI Reduce Numerical (MAX)

Operation: MPI_MAX

Values = {12, 7, 25, 9}

Result:

[
max = 25
]

Quick Comparison Table (Exam Friendly)

Operation	Pattern	Description
MapReduce	Distributed	Big data processing
MPI_Bcast	$1 \rightarrow N$	Same data to all
MPI_Scatter	$1 \rightarrow N$	Split data
MPI_Gather	$N \rightarrow 1$	Collect data
MPI_Reduce	$N \rightarrow 1$	Aggregate result

One-Line Exam Answers

- **MapReduce:** Distributed programming model for processing large datasets.
 - **MPI_Bcast:** Broadcasts identical data from root to all processes.
 - **MPI_Scatter:** Distributes chunks of data to processes.
 - **MPI_Gather:** Collects data from processes to root.
 - **MPI_Reduce:** Performs reduction operation across processes.
-

If you want next:

- ♦ **Short MCQs**
- ♦ **Time complexity numericals**
- ♦ **Hadoop vs MPI comparison**
- ♦ **MPI code snippets (very small)**

Just tell me 👍